MongoDB – Complete Commands & Queries

> Comprehensive guide covering MongoDB shell commands, syntax, examples, and practical use cases for backend development.

Table of Contents

1. Database & Collection Commands
2. Insert Commands (Create)
3. Read Commands (Fetch)
4. Filters & Query Conditions
5. Projection & Field Selection
6. Sorting, Limiting & Pagination
7. Update Commands
8. Delete Commands
9. Indexes
10. Aggregation Pipeline
11. Utility & Debugging

1. Database & Collection Commands

Command	Purpose
`show dbs`	List all databases
`use <dbName>`	Switch to database
`db`	Show current database
`show collections`	List collections

Command	Purpose
`db.createCollection("name")`	Create collection
`db.collection.drop()`	Delete collection

Examples:

```
show dbs
use appDB
db
show collections
db.createCollection("users")
db.users.drop()
```

2. Insert Commands (Create)

insertOne() – Single Document

```
db.users.insertOne({
    name: "Kartik",
    age: 21
})
```

✅ **Use when:** Creating single records

insertMany() – Multiple Documents

```
db.users.insertMany([
    { name: "Alice", age: 20 },
    { name: "Bob", age: 22 }
])
```

✅ **Use when:** Bulk insertion, data seeding

Insert with Custom _id

```
db.users.insertOne({
    _id: 1001,
    name: "External User"
})
```

✅ **Use when:** Mapping external systems, controlled identity

3. Read Commands (Fetch)

find() – Multiple Documents (Cursor)

```
db.users.find()
db.users.find({ age: 21 })
```

findOne() – Single Document

```
db.users.findOne({ name: "Kartik" })
```

Method	Returns	Use Case
`find()`	Cursor (multiple)	Batch operations
`findOne()`	Single document	Get first match

4. Filters & Query Conditions

Equality

```
db.users.find({ age: 21 })
```

Comparison Operators

```
db.users.find({ age: { $gt: 18 } })      // greater than
db.users.find({ age: { $gte: 18 } })     // greater or equal
db.users.find({ age: { $lt: 65 } })      // less than
db.users.find({ age: { $lte: 30 } })     // less or equal
db.users.find({ age: { $ne: 21 } })      // not equal
```

in/nin – Array Matching

```
db.users.find({ age: { $in: [20, 21, 22] } })    // in array
db.users.find({ age: { $nin: [20, 21] } })       // not in array
```

Logical Operators

```
db.users.find({
    $or: [
        { age: 20 },
        { age: 21 }
    ]
})

db.users.find({
    $and: [
        { age: { $gt: 18 } },
        { status: "active" }
    ]
})
```

Array Field Matching

```
db.posts.find({ tags: "mongodb" })         // contains element
db.posts.find({ tags: { $all: ["js", "node"] } })  // contains all
```

5. Projection & Field Selection

Control which fields are returned.

```
db.users.find(
    {},
    { name: 1, email: 1, _id: 0 }
)
```

Value	Effect
`1` or `true`	Include field
`0` or `false`	Exclude field

✅ **Use when:**

- Reducing payload size
- Hiding sensitive fields
- Improving query performance

6. Sorting, Limiting & Pagination

Sorting

```
db.users.find().sort({ age: 1 })       // ascending
db.users.find().sort({ age: -1 })      // descending
db.users.find().sort({ age: -1, name: 1 })  // multiple fields
```

Limiting Results

```
db.users.find().limit(10)
```

Skipping & Pagination

```
db.users.find().skip(10).limit(10)     // page 2 (10 per page)
```

Real-world pagination example:

```
const page = 2;
const limit = 10;
const skip = (page - 1) * limit;

db.users.find().skip(skip).limit(limit)
```

7. Update Commands

updateOne() – First Match

```
db.users.updateOne(
    { name: "Kartik" },
    { $set: { age: 22 } }
)
```

updateMany() – All Matches

```
db.users.updateMany(
    {},
    { $inc: { age: 1 } }
)
```

Update Operators

Operator	Purpose	Example
`$set`	Update fields	`{ $set: { name: "New" } }`
`$inc`	Increment number	`{ $inc: { views: 1 } }`
`$push`	Add to array	`{ $push: { tags: "new" } }`
`$pull`	Remove from array	`{ $pull: { tags: "old" } }`
`$unset`	Remove field	`{ $unset: { temp: "" } }`

Upsert – Insert if Not Exists

```
db.users.updateOne(
    { email: "a@test.com" },
    { $set: { name: "Alice" } },
    { upsert: true }
)
```

8. Delete Commands

deleteOne() – First Match

```
db.users.deleteOne({ name: "Kartik" })
```

deleteMany() – All Matches

```
db.users.deleteMany({ age: { $lt: 18 } })
db.users.deleteMany({})  // ⚠️ Deletes all
```

9. Indexes

Improve query performance on frequently filtered fields.

Create Index

```
db.users.createIndex({ email: 1 })
```

Compound Index

```
db.notes.createIndex({
    userId: 1,
    createdAt: -1
})
```

Manage Indexes

```
db.users.getIndexes()                    // list indexes
db.users.dropIndex({ email: 1 })         // remove index
db.users.dropIndexes()                   // remove all (except _id)
```

10. Aggregation Pipeline

Process and transform data in stages.

Basic Example

```
db.orders.aggregate([
    { $match: { status: "completed" } },
    {
        $group: {
            _id: "$userId",
            total: { $sum: "$amount" },
            count: { $sum: 1 }
        }
    },
    { $sort: { total: -1 } }
])
```

Common Stages

- `$match` – Filter documents
- `$group` – Group by field
- `$sort` – Sort results
- `$limit` – Limit count

- `$project` – Select fields

✅ **Use for:** Reports, analytics, summaries

11. Utility & Debugging

Count Documents

```
db.users.countDocuments()
db.users.countDocuments({ age: { $gt: 18 } })
```

Estimated Count (Faster)

```
db.users.estimatedDocumentCount()
```

Explain Query Performance

```
db.users.find({ age: 21 }).explain("executionStats")
```

✅ **Use when:** Debugging slow queries, optimizing indexes

Quick Reference Cheat Sheet

```
// Create
db.collection.insertOne({})
db.collection.insertMany([])

// Read
db.collection.find()
db.collection.findOne()

// Update
db.collection.updateOne({filter}, {$set: {}})
db.collection.updateMany({filter}, {$inc: {}})

// Delete
db.collection.deleteOne({filter})
db.collection.deleteMany({filter})

// Advanced
db.collection.aggregate([{$match: {}}])
db.collection.createIndex({field: 1})
```